

Docs-as-Code Proof-of-Concept

Tamás Dezső

May 26, 2026

Contents

Introduction	1
Styling	1
Tables	1
Mathematical Notation	2
Code-Blocks	2
C/C++	2
Lua	2
Bash	2
Python	2
Automated Diagrams	3
PlantUML	3
Mermaid	6
Graphviz/DOT	7

Introduction

This document validates the `pandock` build environment. It tests standard markdown rendering, syntax highlighting for various languages, and the automatic vectorization of diagrams.

Pandoc: [The Universal Document Converter](#)

Also check out the [Pandoc Examples](#) for advanced formatting.

Styling

We support standard styling:

- **Bold text** for emphasis.
- *Italic text* for terminology.
- ***Bold-Italic*** for critical notes.
- `Monospaced code` for inline commands.

Tables

Tables are essential for documenting interface parameters or service specifications.

Table 1: Service Interface Specifications

Interface	Protocol	Latency Target	Priority
Gx	Diameter	< 50ms	High
Ro	Diameter	< 100ms	Medium
SIP	UDP/TCP	< 30ms	High

Mathematical Notation

For complex charging algorithms, we use standard LaTeX math support:

The transaction cost C is calculated as:

$$C = \sum_{i=1}^n (t_i \times r_i) + \text{tax}$$

Code-Blocks

By default, syntax highlighting uses the **tango** theme. The font should map to the Google/Ubuntu fonts requested in the YAML frontmatter.

C/C++

```
#include <stdio.h>

typedef struct {
    int transaction_id;
    char status[16];
} Session;

int main() {
    Session s = {1042, "ACTIVE"};
    printf("Session %d is %s\n", s.transaction_id, s.status);
    return 0;
}
```

Lua

```
function check_node(elem)
    if elem.t == "CodeBlock" then
        return pandoc.RawBlock("html", "<!-- Code Block Exists -->")
    end
end
```

Bash

```
#!/bin/bash
echo "Starting compilation..."
mkdir -p build && cd build
cmake .. && make
```

Python

```
def fibonacci(n):
    if n <= 0:
```

```
    return []
elif n == 1:
    return [0]
elif n == 2:
    return [0, 1]
else:
    seq = [0, 1]
    for i in range(2, n):
        seq.append(seq[-1] + seq[-2])
    return seq
```

Automated Diagrams

The blocks below are automatically parsed by the Pandoc diagram Lua filter, rendered via their respective engines, and embedded perfectly into the PDF using XeLaTeX.

PlantUML

- [Website](#)
- [Language Reference Guide](#)
- [Style Guide](#)

This is an example reference to Figure 1.

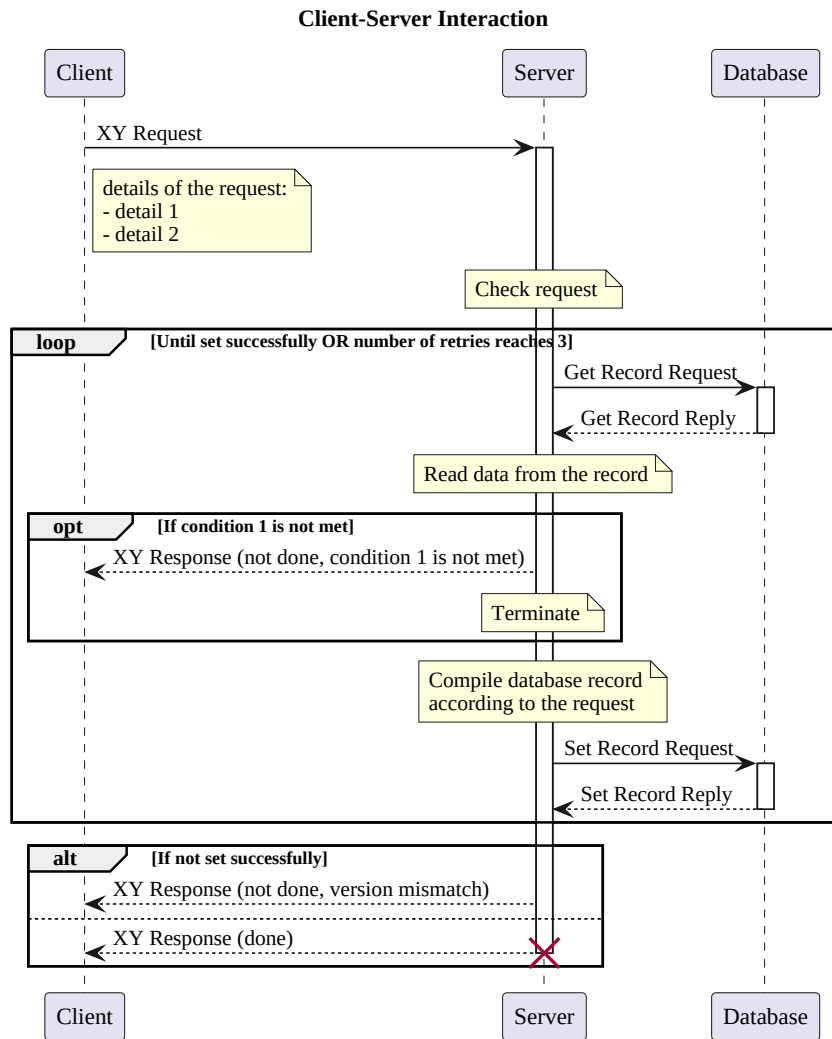


Figure 1: PlantUML Sequence Diagram Example

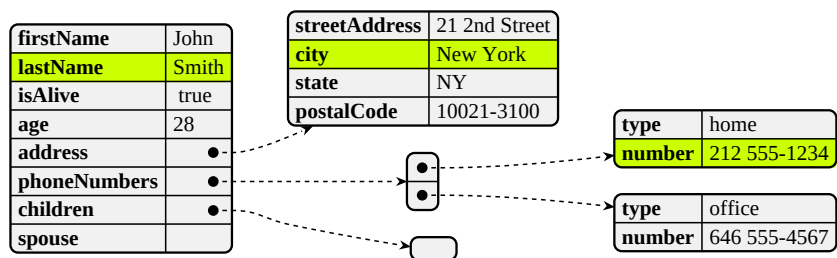


Figure 2: PlantUML Json Diagram Example

Generating Diagrams

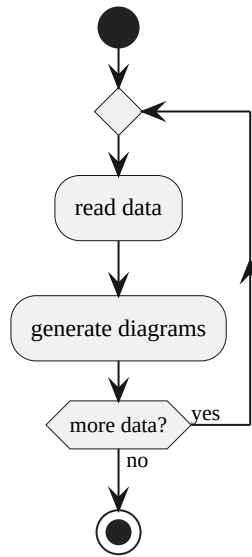


Figure 3: PlantUML Activity Diagram Example

Aaa Class Diagram

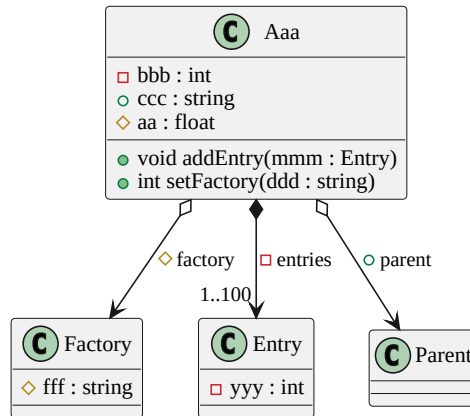


Figure 4: PlantUML Class Diagram Example

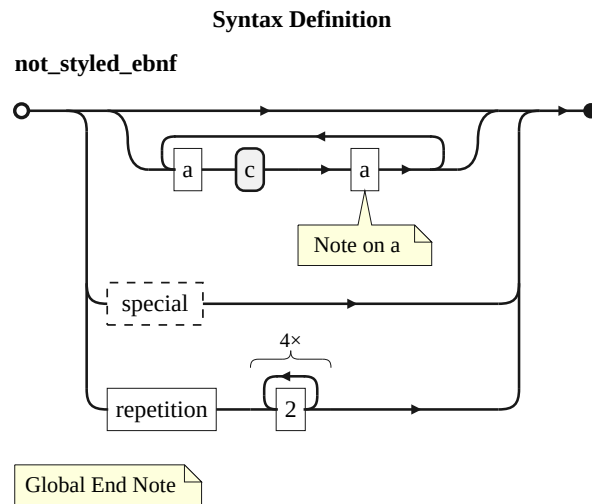


Figure 5: PlantUML EBNF Diagram Example

Mermaid

- [Mermaid Open-Source Intro](#)

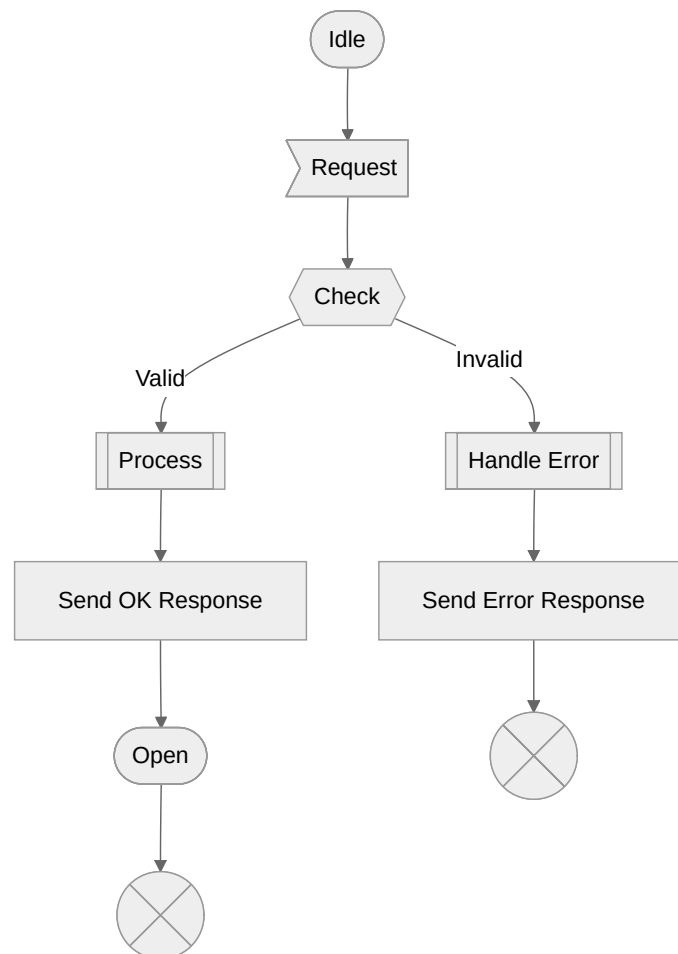


Figure 6: Mermaid Flowchart Example

Graphviz/DOT

- [DOT Language Specification](#)
- [Gallery](#)

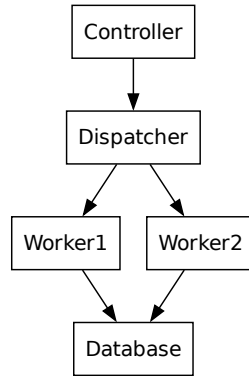


Figure 7: Graphviz/DOT Diagram Example